TERM PAPER

Learning-Augmented OS Heuristics: Classical Algorithms Meet Adaptive Prediction

Submitted By

Md. Fojla Rabbi
ID: 202414067

CSE-307: Operating Systems

Section: B

Learning-Augmented OS Heuristics: Classical Algorithms Meet Adaptive Prediction

CSE-307 Operating Systems — Track 1: Learned Page Replacement

1. Problem Framing

Page replacement determines which resident page should be removed when a page fault occurs and all available frames are occupied. This experiment compares three classical policies—FIFO, LRU, and Optimal (Belady)—with a lightweight learned policy. The official brief asks for a synthetic access trace whose behavior changes partway through, followed by measurement of page faults and hit ratios before and after the shift.

FIFO evicts the page that has been resident for the longest time, whereas LRU uses recent access history. The Optimal policy uses knowledge of future references and therefore serves as an offline benchmark rather than a practical online policy. The OS concepts framing follows Silberschatz, Galvin, and Gagne [1].

2. Implementation

The implementation uses six page frames and a 50-page address space. FIFO maintains a queue, LRU maintains the most recent access position of each resident page, and Optimal examines future references to choose the page whose next use is farthest away. The learned policy is a Decision Tree classifier. For each candidate victim at a page fault, it receives three features: recency since the candidate's last access, frequency in a recent 20-reference window, and observed global frequency. The model is trained from Optimal-policy labels on separate locality-style traces. DecisionTreeClassifier is provided by scikit-learn [2].

Future references are used to construct training labels but are not supplied as input features when the learned policy makes an eviction decision.

3. Experimental Setup

The test trace contains 4,000 references. Phase 1 contains 2,000 locality-heavy/sequential accesses: most references repeatedly visit a small six-page hot set, with occasional accesses to colder pages. Phase 2 contains 2,000 random/bursty accesses over a larger page universe. The same deterministic test trace (seed 307) is supplied to all four policies. The learned model uses max_depth=5 and random_state=307.

4. Results

Policy	Faults	Hit ratio	P1 faults	P1 hit	P2 faults	P2 hit
FIFO	2614	0.346	904	0.548	1710	0.145
LRU	2152	0.462	434	0.783	1718	0.141
Optimal	1597	0.601	351	0.825	1246	0.377
Learned-DecisionTree	2152	0.462	387	0.806	1765	0.117

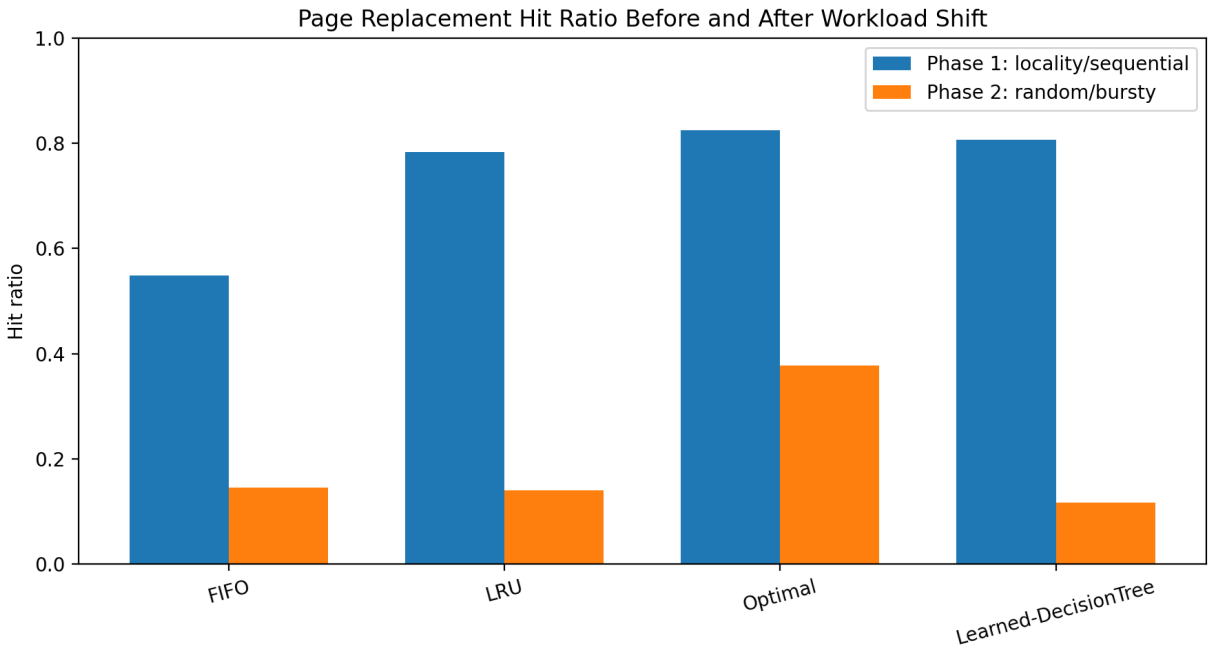


Figure 1. Hit ratio before and after the workload shift. This is a reproducible reference run; rerun the script and verify the CSV/figure before submission.

5. Analysis

The workload shift has a clear effect on the measured behavior. In Phase 1, where references are concentrated around a small working set, LRU reaches a 0.783 hit ratio in the reference run, compared with 0.548 for FIFO and 0.825 for Optimal. After the shift to random/bursty accesses, all policies have lower hit ratios because the working set becomes larger and less predictable. Optimal remains an offline benchmark, while the learned policy falls from 0.806 in Phase 1 to 0.117 in Phase 2.

The learned policy has 2,152 total faults in this reference run, equal to LRU, but its phase behavior differs: it performs relatively well during the locality-heavy phase and degrades strongly after the distribution changes. This illustrates the central motivation of the brief: a lightweight learned layer can exploit observable workload features, but a model trained mainly on one workload regime may generalize poorly after an access-pattern shift.

These results are one controlled synthetic experiment, not a universal ranking of page-replacement policies. The purpose is to connect measured behavior to memory-management concepts and demonstrate the effect of a deliberate workload shift.

6. References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018, ISBN 978-1-119-32091-3.

[2] Scikit-learn developers, “DecisionTreeClassifier,” scikit-learn documentation.

AI assistance disclosure: ChatGPT was used for implementation assistance and project-structure guidance. The student should verify the code, rerun the experiment, and confirm the final interpretation before submission, as required by the course brief.